

Static Uneven Distribution Menggunakan Multithreading Python

Nama : Edsel Sulthan

NRP : 152024166

Implementasi Static Uneven Distribution pada Multithreading Python

1. Dasar Teori

1.1 Multithreading

Multithreading adalah teknik menjalankan beberapa proses ringan (thread) secara bersamaan di dalam satu program. Pada Python, multithreading dapat dibuat menggunakan modul `threading`.

Keuntungan multithreading:

- Mempercepat eksekusi program tertentu.
- Memungkinkan beberapa tugas berjalan secara paralel.
- Efisien untuk simulasi proses I/O dan task independen.

1.2 Static Distribution

Static distribution adalah metode pembagian tugas dimana jumlah pekerjaan setiap thread sudah ditentukan sejak awal program dijalankan.

Contoh:

- Thread-0 → 6 tugas
- Thread-1 → 4 tugas
- Thread-2 → 2 tugas

1.3 Uneven Distribution

Uneven distribution berarti pembagian tugas tidak merata antar thread.

Pada praktikum ini:

- Total proses = 12
- Distribusi:
 - Thread-0 = 6 tugas
 - Thread-1 = 4 tugas
 - Thread-2 = 2 tugas

Karena jumlah tugas berbeda, maka waktu selesai tiap thread juga berbeda.

2. Listing Program

```
import threading
import time

# NRP: 166
# digit genap dari NRP = 6, 6
# total proses = 6 + 6 = 12
# distribusi static uneven ke 3 thread: 6, 4, 2

tugas_thread = [6, 4, 2]

waktu_selesai = [0, 0, 0]

def kerjain(id_thread, jumlah_tugas):
    print(f"thread-{id_thread} mulai, bagian {jumlah_tugas} tugas")

    total = 0
    for i in range(jumlah_tugas):
        time.sleep(0.5)
        total += 1
        print(f"  thread-{id_thread} selesai tugas ke-{i+1}")

    waktu_selesai[id_thread] = total * 0.5
    print(f"thread-{id_thread} beres semua, total waktu estimasi: {waktu_selesai[id_thread]}s")

t0 = threading.Thread(target=kerjain, args=(0, tugas_thread[0]))
t1 = threading.Thread(target=kerjain, args=(1, tugas_thread[1]))
t2 = threading.Thread(target=kerjain, args=(2, tugas_thread[2]))

mulai = time.time()

t0.start()
t1.start()
t2.start()

t0.join()
t1.join()
t2.join()

selesai = time.time()
total_waktu = round(selesai - mulai, 2)

print()
print("=== HASIL ===")
```

```

print(f"distribusi tugas : {tugas_thread}")
print(f"waktu tiap thread : {[f'{t}s' for t in waktu_selesai]}")
print(f"wall clock time : {total_waktu}s")

maks = max(waktu_selesai)
mini = min(waktu_selesai)
selisih = round(maks - mini, 2)

print(f"selisih waktu : {selisih}s")

if selisih <= 0.1:
    print("distribusi IDEAL tercapai!")
else:
    print(f"distribusi belum ideal, selisih {selisih}s")

```

3. Penjelasan Program

3.1 Import Library

```

import threading
import time

```

- `threading` digunakan untuk membuat thread.
- `time` digunakan untuk simulasi waktu kerja.

3.2 Pembagian Tugas

```
tugas_thread = [6, 4, 2]
```

Artinya:

- Thread-0 mengerjakan 6 tugas.
- Thread-1 mengerjakan 4 tugas.
- Thread-2 mengerjakan 2 tugas.

Distribusi ini disebut static uneven karena:

- Ditentukan dari awal.
- Tidak merata.

3.3 Fungsi Kerja Thread

```
def kerjain(id_thread, jumlah_tugas):
```

Fungsi ini dijalankan oleh masing-masing thread.

Setiap tugas disimulasikan menggunakan:

```
time.sleep(0.5)
```

yang berarti setiap task membutuhkan waktu 0.5 detik.

3.4 Pembuatan Thread

```
t0 = threading.Thread(...)
```

Program membuat 3 thread:

- t0
- t1
- t2

Masing-masing memiliki jumlah tugas berbeda.

3.5 Menjalankan Thread

```
t0.start()
```

Thread dijalankan secara paralel menggunakan `.start()`.

3.6 Sinkronisasi Thread

```
t0.join()
```

`.join()` digunakan agar program utama menunggu seluruh thread selesai.

4. Hasil Eksekusi

Contoh hasil program:

```
thread-0 mulai, sebagian 6 tugas  
thread-1 mulai, sebagian 4 tugas  
thread-2 mulai, sebagian 2 tugas
```

```
thread-2 beres semua, total waktu estimasi: 1.0s  
thread-1 beres semua, total waktu estimasi: 2.0s  
thread-0 beres semua, total waktu estimasi: 3.0s
```

```
=== HASIL ===
distribusi tugas : [6, 4, 2]
waktu tiap thread : ['3.0s', '2.0s', '1.0s']
wall clock time   : 3.0s
selisih waktu     : 2.0s
distribusi belum ideal
```

5. Analisis Hasil

Dari hasil eksekusi diperoleh:

Thread	Jumlah Tugas	Waktu
Thread-0	6	3.0 s
Thread-1	4	2.0 s
Thread-2	2	1.0 s

Karena setiap tugas membutuhkan 0.5 detik:

- Thread dengan tugas lebih banyak membutuhkan waktu lebih lama.
- Thread-0 menjadi bottleneck karena memiliki beban terbesar.

Analisis Wall Clock Time

Wall clock time total $\approx$ 3 detik.

Hal ini terjadi karena:

- Program selesai ketika thread paling lambat selesai.
- Thread-0 membutuhkan waktu paling lama.

Analisis Ketidakseimbangan

Selisih waktu:

```
3.0 - 1.0 = 2.0 detik
```

Karena selisih lebih dari 0.1 detik:

- Distribusi dianggap tidak ideal.
- Beban kerja antar thread tidak seimbang.

6. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Static uneven distribution membagi tugas secara tetap namun tidak merata.
2. Thread dengan tugas lebih banyak membutuhkan waktu eksekusi lebih lama.
3. Total waktu program dipengaruhi oleh thread paling lambat.
4. Distribusi yang tidak seimbang menyebabkan efisiensi paralelisme menurun.
5. Untuk mendapatkan performa optimal, pembagian tugas sebaiknya lebih merata.